Plateforme Alumni UASZ

Documentation d'architecture logicielle
Stack, diagrammes UML et justification des choix

GL5121 — Architecture Logicielle · Livrables n°5 et n°6

Master Informatique — Université Assane Seck de Ziguinchor

Rédaction : Cheikh Omar Ndiaye

Novembre 2026

Sommaire

1. **Partie I — Architecture : exigences, choix et justification**
    - Architecture logicielle : choix et justification
2. **Partie II — Décisions d'architecture (ADR)**
    - Architecture Decision Records (ADR)
    - ADR-0001 : Architecture en microservices plutôt qu'un monolithe modulaire
    - ADR-0002 : Approche Contract First avec OpenAPI et SPIs générés
    - ADR-0003 : Monorepo unique pour les services, les contrats et l'infrastructure
    - ADR-0004 : Une base PostgreSQL par service, schéma géré par Flyway
    - ADR-0005 : Authentification stateless par JWT signé avec un secret partagé (HS256)
    - ADR-0006 : Communication entre services en REST synchrone (pas de broker en v1)
    - ADR-0007 : Distribution des SPIs par un dépôt Maven versionné dans Git
    - ADR-0008 : Conteneurisation Docker et déploiement Kubernetes (kustomize, kind en local)
    - ADR-0009 : Java 21 et Spring Boot comme stack backend commune
3. **Partie III — Diagrammes UML exigés**
    - Diagramme de composants
    - Diagramme de packages
    - Diagramme de déploiement
    - Diagrammes de classes (par service)
4. **Partie IV — Diagrammes complémentaires**
    - Diagrammes de cas d'utilisation (par rôle)
    - Diagrammes de séquence des flux clés

Partie I — Architecture : exigences, choix et justification

Architecture logicielle : choix et justification

> Livrables n°5 et n°6 du sujet GL5121 (Architecture Logicielle). Ticket Jira : *Documentation d'architecture : stack, diagrammes UML et justification des choix* (epic AL-2). Les diagrammes UML sont dans `diagrams/` (index : `README.md`). Chaque décision est détaillée dans un ADR : `adr/`.

Sommaire

1. Contexte
2. Exigences prises en compte
3. Monolithe modulaire ou microservices
4. Stack technique retenue
5. Composants logiciels et interfaces
6. Sécurité
7. Déploiement, CI/CD et qualité
8. Points ouverts

---

1. Contexte

La plateforme **Alumni UASZ** met en relation les étudiants du Master Informatique de l'UASZ, les anciens diplômés (alumni), les enseignants, les entreprises partenaires et l'administration. Elle doit permettre :

- la gestion des comptes et l'authentification ;
- la gestion des profils (étudiant, alumni), des CV, des entreprises et des promotions ;
- la publication d'offres d'emploi ou de stage, les candidatures et les propositions directes ;
- le matching automatique entre étudiants et offres, des statistiques et des rapports pour l'administration ;
- la messagerie privée, les groupes de discussion et les notifications.

Le backend sert une application front (équipe front) via des API REST.

2. Exigences prises en compte

> Le **cahier des besoins** (livrable n°1, ticket **AL-62**) fait référence pour le détail des exigences. Il sera déposé dans `docs/`. Cette section reprend sa structure (acteurs, exigences fonctionnelles par domaine, exigences non fonctionnelles, contraintes) et indique, pour chaque exigence, comment l'architecture y répond.

Problème à résoudre : aujourd'hui, les échanges entre étudiants, alumni, enseignants et entreprises sont dispersés sur WhatsApp et les réseaux sociaux. Il n'existe ni annuaire fiable, ni suivi des offres et des candidatures, ni indicateur d'insertion professionnelle.

2.1 Acteurs

Acteur	Rôle technique (`RoleName`)	Remarque
Étudiant	`ETUDIANT`	
Alumni	`ALUMNI`	
Enseignant	`ENSEIGNANT`	
Administrateur	`ADMINISTRATEUR`	
Entreprise partenaire	**absent**	Acteur du cahier des besoins, mais sans rôle dans le contrat `user-service` v1.0.0 (voir §8)

2.2 Exigences fonctionnelles (par domaine)

ID	Domaine	Exigence	Acteurs	Service
EF-01	Utilisateurs	Inscription, connexion, refresh/logout, confirmation email, mot de passe oublié	Tous	`user-service`
EF-02	Utilisateurs	Gestion des utilisateurs, de leur statut (activation, suspension) et des rôles	Administrateur	`user-service`
EF-03	Promotions	Promotions académiques et liste des diplômés	Administrateur, Enseignant	`alumni-student-company-service`
EF-04	CV	Profil étudiant (spécialité, compétences, disponibilité) et CV structuré	Étudiant	`alumni-student-company-service`
EF-05	Alumni	Profil alumni et parcours professionnel (expériences)	Alumni	`alumni-student-company-service`
EF-06	Alumni	Recommandation d'un étudiant auprès d'une entreprise	Alumni, Enseignant	`alumni-student-company-service`
EF-07	Entreprises	Annuaire et fiches des entreprises partenaires	Tous ; Administrateur et Entreprise en écriture	`alumni-student-company-service`
EF-08	Offres	Publication, recherche, mise à jour et clôture d'offres d'emploi ou de stage	Alumni, Entreprise, Administrateur	`job-offer-service`
EF-09	Offres	Candidature à une offre et suivi de son état	Étudiant ; recruteur pour le changement d'état	`job-offer-service`
EF-10	Offres	Proposition directe d'une offre à un étudiant, puis acceptation ou refus	Alumni, Entreprise ; Étudiant	`job-offer-service`
EF-11	Offres	Matching étudiant ↔ offre, avec score et compétences manquantes	Étudiant, recruteur	`analytics-matching-service`
EF-12	Messagerie	Messagerie privée 1-à-1	Tous	`notification-messaging-service`
EF-13	Messagerie	Groupes de discussion (création, membres, messages)	Tous	`notification-messaging-service`
EF-14	Messagerie	Notifications in-app, email et push (lues / non lues)	Tous	`notification-messaging-service`
EF-15	Tableau de bord	Statistiques globales, rapport d'insertion, écart de compétences	Administrateur	`analytics-matching-service`

2.3 Exigences non fonctionnelles

ID	Exigence	Réponse architecturale
ENF-01	**Sécurité** : JWT, mots de passe hachés, RBAC	JWT signé (HMAC) émis par `user-service` et vérifié localement par chaque service ; mots de passe hachés avec **BCrypt**, jamais stockés ni journalisés en clair ; contrôle d'accès par rôle (`ETUDIANT` , `ALUMNI` , `ENSEIGNANT` , `ADMINISTRATEUR`) dans chaque service ; API stateless ; secrets hors du dépôt (voir §6)
ENF-02	**Performance**	Pagination de toutes les listes (`page` , `size`) ; vérification du JWT sans appel réseau ; index sur les colonnes de recherche ; champs d'affichage dénormalisés (`companyName` , `studentName`) pour éviter des appels en cascade ; calculs lourds (matching, rapports) isolés dans un service dédié
ENF-03	**Disponibilité**	2 réplicas par service en Kubernetes, rolling update sans indisponibilité (`maxUnavailable: 0`), sondes liveness/readiness, arrêt gracieux ; une panne reste limitée à un service
ENF-04	**Évolutivité** (montée en charge et fonctionnelle)	Services sans état, scalables indépendamment ; une base par service ; nouveaux domaines ajoutés comme nouveaux services, sans toucher aux autres
ENF-05	**Maintenabilité**	Découpage par domaine métier ; même structure en couches dans chaque service ; contrats OpenAPI versionnés et code d'interface généré (SPI) ; SonarQube
ENF-06	**Ergonomie**	Côté backend : API REST cohérentes (mêmes conventions, pagination et format d'erreur `ErrorResponse` uniques) et messages d'erreur explicites, pour que le front puisse offrir une interface simple. La conception de l'interface relève de l'équipe front
ENF-07	**Compatibilité mobile**	API REST/JSON indépendantes du client, donc utilisables par le web responsive comme par une future application mobile ; JWT adapté aux clients mobiles (pas de cookie de session) ; canal de notification `PUSH` prévu dans le contrat ; réponses paginées et légères
ENF-08	**RGPD / données personnelles**	Données personnelles cantonnées à leur service propriétaire et référencées ailleurs par UUID ; accès authentifié et filtré par rôle ; mots de passe hachés ; consentement via la confirmation de l'email ; droit de suppression avec `DELETE /users/{id}` ; HTTPS de bout en bout via l'ingress. **À compléter** : durée de conservation, export des données personnelles, suppression propagée aux autres services
ENF-09	**Intégrité des données**	PostgreSQL transactionnel ; schéma versionné par Flyway ; `ddl-auto: validate`
ENF-10	**Observabilité** (minimum)	Spring Boot Actuator (`health` , `info`) ; à compléter (logs centralisés, métriques)

2.4 Contraintes

Contrainte	Mise en œuvre dans le projet
Contract First	Contrats OpenAPI dans `contracts/` ; interfaces et modèles générés (`spis/`) et implémentés par les contrôleurs ; toute évolution passe par une PR avec un incrément de version
Git Flow	Branche d'intégration `develop` , branche de production `main` , branches `feature/AL-xx-...` par ticket Jira, fusion par Pull Request
Docker	`docker/service.Dockerfile` partagé, docker-compose par service pour le développement local
CI/CD	GitHub Actions : `ci.yml` (vérification des SPIs, build des services modifiés), `deploy.yml` (images GHCR, déploiement Kubernetes)
SonarQube	`sonar/` en local et `sonar.yml` en CI pour les 5 services

3. Monolithe modulaire ou microservices

3.1 Les deux options

Monolithe modulaire : une seule application déployable, découpée en modules internes aux frontières nettes (un module par domaine), qui partagent un processus et en général une base de données.

Microservices : plusieurs applications déployées indépendamment, chacune propriétaire d'un domaine et de ses données. Elles communiquent par le réseau à travers des contrats d'API.

3.2 Comparaison

Critère	Monolithe modulaire	Microservices	Avantage
Simplicité de développement initial	Un seul projet, appels de méthode, une transaction	Plusieurs projets, appels réseau, cohérence à gérer entre services	Monolithe
Déploiement	Un seul artefact : simple, mais tout est redéployé à chaque changement	Déploiement indépendant de chaque service	Microservices
Scalabilité	Toute l'application est dupliquée	Seul le service chargé est dupliqué (ex. messagerie)	Microservices
Isolation des pannes	Une fuite mémoire ou un crash touche tout le système	Une panne reste contenue dans un service	Microservices
Autonomie des équipes	Code partagé, conflits fréquents, release commune	Chaque équipe possède un service et son contrat	Microservices
Cohérence des données	Transactions ACID entre modules	Cohérence à terme entre services, données dupliquées ou référencées par ID	Monolithe
Coût d'infrastructure	Faible (1 application + 1 base)	Plus élevé (N applications, N bases, orchestrateur)	Monolithe
Observabilité et débogage	Pile d'appels locale	Traçage distribué nécessaire	Monolithe
Frontières entre domaines	Respectées par discipline seulement, faciles à contourner	Imposées physiquement par le réseau et les contrats	Microservices
Valeur pédagogique (sujet GL5121)	Faible pour les sujets DevOps et distribués	Contrats, conteneurs, orchestration, CI/CD	Microservices

3.3 Choix retenu : microservices

> Décision formalisée dans ADR-0001.

Nous retenons une architecture en **5 microservices**. Raisons :

1. **Des domaines métier bien séparés.** Identité, profils, offres, analyse et communication ont peu de règles en commun et des cycles de vie différents. Le découpage suit ces domaines (*bounded contexts*).
2. **Des charges différentes.** La messagerie et les notifications reçoivent beaucoup de petites requêtes. Le matching et les rapports font des calculs ponctuels et coûteux. Les microservices permettent de

dimensionner chacun séparément.
3. **Le travail en équipes parallèles.** Le projet est réalisé par plusieurs équipes (back, front, devops). Les contrats OpenAPI servent de frontière stable : chaque équipe avance sur son service, et le front peut développer avec des mocks générés depuis les contrats.
4. **Des pannes isolées.** Une panne de la messagerie ne doit pas empêcher un étudiant de postuler.
5. **Les attentes du sujet.** Le sujet demande de mettre en œuvre contrats d'interface, conteneurisation, orchestration et CI/CD. Ces sujets prennent tout leur sens dans une architecture distribuée.

Coûts acceptés, et comment nous les limitons :

Coût	Mesure
Complexité de l'outillage	**Monorepo**, `Makefile` et scripts identiques dans chaque service, Dockerfile unique partagé
Dérive des interfaces	Approche **contract first** : SPIs générés et versionnés ; une modification de contrat passe par une PR
Données réparties	Chaque service est propriétaire de ses données ; les autres y font référence par UUID, et les champs d'affichage (`companyName` , `studentName` ...) sont dénormalisés
Authentification distribuée	JWT vérifié localement par chaque service, sans appel à `user-service` à chaque requête
Coût d'exploitation	Kubernetes local avec **kind** ; une seule image paramétrée par `SERVICE`

> Le **monolithe modulaire** restait une option défendable pour une petite équipe. Le découpage par paquets et par domaine utilisé ici permettrait d'ailleurs de regrouper les services en un monolithe si le coût d'exploitation devenait trop élevé. L'inverse, extraire des services d'un monolithe, est plus difficile.

4. Stack technique retenue

> Voir les ADR 0002 à 0009.

Couche	Choix	Justification
Langage	**Java 21 (LTS)**	Support long terme, *records*, *virtual threads* ; langage maîtrisé par l'équipe et enseigné dans le cursus
Framework	**Spring Boot 4.1** (Web MVC, Validation, Security, Data JPA, Actuator)	Standard de l'industrie pour les microservices Java ; écosystème complet (sécurité, persistance, santé) ; configuration externalisée
Contrats d'API	**OpenAPI 3** + **openapi-generator**	Contract first : le contrat est la source de vérité ; interfaces et DTO générés (SPI), donc pas d'écart entre la documentation et le code
Distribution des SPIs	Dépôt Maven versionné dans Git (`spis/repo/`)	Aucun registre à héberger ; un `git pull` suffit ; versions immuables
Base de données	**PostgreSQL 16**, une base par service	SGBD relationnel open source, fiable et transactionnel ; *database per service* pour l'autonomie des services
Migrations	**Flyway**	Schéma versionné et rejoué à l'identique dans chaque environnement ; JPA en mode `validate`

Couche	Choix	Justification
Sécurité	**Spring Security** + **JWT** (secret HMAC partagé, émetteur `alumni-uasz-user`)	Stateless, donc compatible avec la mise à l'échelle horizontale ; vérification locale sans appel réseau
Build	**Maven** (wrapper `mvnw`)	Standard Spring ; build reproductible sans installation préalable
Conteneurisation	**Docker** (build multi-étapes, image JRE, utilisateur non root)	Image légère et sûre ; un seul `service.Dockerfile` paramétré par `SERVICE`
Dev local	**docker-compose** par service, `make setup / up`	Démarrer un service et sa base en une commande
Orchestration	**Kubernetes** + **kustomize** (overlays dev/prod), **kind** en local, ingress **NGINX**	Rolling updates, sondes de santé, réplicas ; même manifeste du poste de développement à la production
CI/CD	**GitHub Actions** (CI sélective, CD vers GHCR et K8s)	Intégré au dépôt ; seuls les services modifiés sont construits
Qualité	**SonarQube**	Bugs, vulnérabilités, *code smells* et couverture pour les 5 services

5. Composants logiciels et interfaces

Voir le diagramme de composants et le diagramme de packages.

5.1 Composants

Composant	Port	Responsabilité	Interface fournie (contrat)	Préfixe exposé (ingress)
`user-service`	8081	Comptes, authentification, rôles ; **émet les JWT**	`contracts/user-service.yaml` (14 opérations)	`/user-service/v1`
`alumni-student-company-service`	8082	Étudiants, CV, alumni, expériences, recommandations, entreprises, promotions	`contracts/alumni-student-company-service.yaml` (18)	`/alumni-student-company-service/v1`
`job-offer-service`	8083	Offres, candidatures, propositions	`contracts/job-offer-service.yaml` (13)	`/job-offer-service/v1`
`analytics-matching-service`	8084	Matching, statistiques, rapports	`contracts/analytics-matching-service.yaml` (5)	`/analytics-matching-service/v1`
`notification-messaging-service`	8085	Messagerie privée, groupes, notifications	`contracts/notification-messaging-service.yaml` (20)	`/notification-messaging-service/v1`
SPIs (`spis/`)		Interfaces `*Api` et modèles générés depuis les contrats	Artefacts Maven `com.uasz.alumni:*-spi:1.0.0`	
Ingress NGINX	80/443	Point d'entrée unique, routage par préfixe		`https://api.alumni-uasz.sn/<service>/v1`

5.2 Interfaces entre composants

- **Front** → **services** : REST/JSON via l'ingress, avec un en-tête `Authorization: Bearer <JWT>` (sauf pour les endpoints publics : `/auth/*`, consultation et recherche d'offres).
- **Services** → **services** : REST/JSON synchrone, avec propagation du JWT de l'appelant. Les dépendances prévues sont les suivantes :

Consommateur	Fournisseur	Usage
`analytics-matching-service`	`alumni-student-company-service`	Compétences des étudiants, diplômés, promotions
`analytics-matching-service`	`job-offer-service`	Offres ouvertes et compétences requises
`job-offer-service`	`alumni-student-company-service`	Nom de l'entreprise, profil et CV du candidat
`job-offer-service`, `analytics-matching-service`, `user-service`	`notification-messaging-service`	`POST /notifications/send` (changement d'état d'une candidature, proposition, nouvelle offre adaptée, confirmation de compte)
tous	`user-service`	Aucun appel à l'exécution : le JWT est vérifié localement avec le secret partagé

- **Données** : chaque service est le **seul** à accéder à sa base. Les autres données sont référencées par **UUID** (`userId`, `companyId`, `offerId`, `studentId`…), jamais par clé étrangère entre bases.
- **Évolution des contrats** : modifier `contracts/*.yaml` et `CHANGELOG.md`, incrémenter la version des SPIs, régénérer, aligner `alumni.spi.version` dans les services concernés, le tout dans une Pull Request.

5.3 Structure interne d'un service

Chaque service suit la même organisation en couches (package racine `com.uasz.alumni.<service>`) :

```
controller   -> implémente les interfaces SPI générées (spi.api.*Api)
service      -> logique métier, transactions
repository   -> Spring Data JPA
model        -> entités JPA
dto          -> mapping entités <-> modèles SPI (spi.model.*)
security/jwt -> filtre et validation du JWT
config       -> sécurité, préfixe API
exceptions   -> ApiException, GlobalExceptionHandler -> ErrorResponse
utils
```

6. Sécurité

- **Authentification** : `user-service` vérifie les identifiants et émet un *access token* JWT (court) et un *refresh token*. Les 4 autres services vérifient la signature et l'émetteur avec le même secret (`JWT_SECRET`, Base64 de 32 octets minimum).
- **Autorisation** : le rôle (`ETUDIANT`, `ALUMNI`, `ENSEIGNANT`, `ADMINISTRATEUR`) est porté par le JWT et contrôlé dans chaque service.
- **Stateless** : pas de session HTTP ; CSRF, formulaire de connexion et HTTP Basic désactivés.
- **Secrets** : fichier `.env` local non versionné et généré par `make setup` ; *Secrets* Kubernetes en cluster.

- **Conteneurs** : utilisateur non root (UID 10001), profil seccomp `RuntimeDefault`, limites CPU et mémoire.
- **Exposition** : seules les sondes `health` et `info` d'Actuator sont exposées, sans détails.

7. Déploiement, CI/CD et qualité

Voir le diagramme de déploiement.

- **Kubernetes** : namespace `alumni-uasz`. Par service : un `Deployment` à 2 réplicas, un `Service`, une `ConfigMap` et des `Secrets`, plus un `StatefulSet` PostgreSQL en dev (stockage de 1 Gi). L'ingress NGINX route chaque préfixe vers son service.
- **CI** (`.github/workflows/ci.yml`) : vérifie que les contrats génèrent des SPIs valides, détecte les services modifiés et ne construit qu'eux.
- **CD** (`deploy.yml`) : build Maven, image poussée sur **GHCR**, déploiement kustomize.
- **Qualité** (`sonar.yml` et `sonar/`) : analyse SonarQube des 5 services sur `main` et `develop`.

8. Points ouverts

Sujet	Constat	Piste
Rôle « Entreprise partenaire »	Acteur du cahier des besoins (AL-62), et les contrats parlent de propositions « par un Alumni ou Entreprise », mais `RoleName` ne contient pas `ENTREPRISE`	Ajouter `ENTREPRISE` à `RoleName` et relier le compte à une `Company` (contrat `user-service` 1.1.0, à décider en équipe)
RGPD	Pas encore d'export des données personnelles ni de règles de conservation ; la suppression d'un compte n'est pas propagée aux autres services	Endpoint d'export, durée de conservation, événement `UserDeleted` traité par chaque service
Communication asynchrone	Les notifications passent par un appel REST synchrone	Introduire un broker (RabbitMQ ou Kafka) pour les événements (`ApplicationStatusChanged`, `OfferPublished` …)
API Gateway	L'ingress fait seulement du routage	Gateway (Spring Cloud Gateway) pour la limitation de débit et la vérification centralisée du JWT, si besoin
Observabilité	Seul Actuator `health/info` est exposé	Prometheus et Grafana, logs centralisés, traçage (OpenTelemetry)
Stockage des fichiers	`CvResponse.fileUrl`, `logoUrl`	Stockage objet (MinIO ou S3)

Partie II — Décisions d'architecture (ADR)

Architecture Decision Records (ADR)

Un ADR consigne **une** décision d'architecture importante : son contexte, les options étudiées, la décision et ses conséquences. Il permet à un nouveau membre (ou au jury) de comprendre **pourquoi** le système est ainsi, pas seulement **comment** il est fait.

Règles

- Un fichier par décision : `NNNN-titre-court.md`, numéroté dans l'ordre chronologique.
- Un ADR **accepté n'est jamais réécrit**. Si la décision change, on crée un nouvel ADR qui le **remplace** (`Remplacé par ADR-00XX`), et on met à jour le statut de l'ancien.
- Statuts : `Proposé` → `Accepté` → (`Déprécié` | `Remplacé par ADR-XXXX`).
- Toute création ou modification passe par une Pull Request relue par l'équipe.
- Modèle : `0000-modele.md`.

Index

N°	Décision	Statut	Date
0001	Architecture en microservices plutôt qu'un monolithe modulaire	Accepté	2026-11-01
0002	Approche Contract First avec OpenAPI et SPIs générés	Accepté	2026-11-01
0003	Monorepo unique pour les services, les contrats et l'infrastructure	Accepté	2026-11-01
0004	Une base PostgreSQL par service, schéma géré par Flyway	Accepté	2026-11-01
0005	Authentification stateless par JWT signé avec un secret partagé (HS256)	Accepté, à réévaluer	2026-11-01
0006	Communication entre services en REST synchrone (pas de broker en v1)	Accepté, à réévaluer	2026-11-07
0007	Distribution des SPIs par un dépôt Maven versionné dans Git	Accepté	2026-11-01
0008	Conteneurisation Docker et déploiement Kubernetes (kustomize, kind en local)	Accepté	2026-11-01
0009	Java 21 et Spring Boot comme stack backend commune	Accepté	2026-11-01

ADR-0001 : Architecture en microservices plutôt qu'un monolithe modulaire

- **Statut** : Accepté
- **Date** : 2026-11-01
- **Décideurs** : équipes back, devops
- **Ticket** : AL-2 (socle technique)

Contexte

La plateforme couvre cinq domaines aux règles et aux cycles de vie différents : identité, profils (étudiants, alumni, entreprises), offres et candidatures, analyse et matching, communication. Le projet est réalisé par plusieurs équipes en parallèle (back, front, devops) sur un temps court. Le sujet GL5121 demande de mettre en œuvre des contrats d'interface, la conteneurisation, l'orchestration et la CI/CD.

La volumétrie attendue est modeste : quelques centaines à quelques milliers d'utilisateurs, ceux d'un master universitaire et de ses diplômés.

Options étudiées

1. **Monolithe modulaire** : une application Spring Boot, un module par domaine, une base.
    - Avantages : simple à développer, à tester et à déboguer ; transactions ACID entre domaines ; coût d'infrastructure minimal.
    - Inconvénients : frontières entre modules faciles à contourner ; un seul déploiement pour tout ; conflits entre équipes sur un même code ; met peu en pratique les sujets distribués du cours.
2. **Microservices** : un service déployable par domaine, chacun avec sa base, reliés par des contrats.
    - Avantages : autonomie des équipes ; déploiement, montée en charge et pannes isolés par service ; frontières imposées physiquement.
    - Inconvénients : complexité opérationnelle (N builds, N bases, orchestrateur) ; cohérence des données à terme ; débogage distribué.

Décision

Nous retenons **5 microservices** découpés par domaine métier : `user`, `alumni-student-company`, `job-offer`, `analytics-matching` et `notification-messaging`.

Ce qui fait pencher la balance, honnêtement : la volumétrie seule ne justifie pas des microservices. Un monolithe modulaire tiendrait la charge sans difficulté. Le choix repose sur trois motifs :

- **organisationnel** : plusieurs équipes doivent avancer en parallèle sans se bloquer, avec des contrats comme frontière ;
- **pédagogique** : le sujet exige de pratiquer l'architecture distribuée ;
- **évolutif** : les domaines ont des profils de charge différents. La messagerie reçoit beaucoup de petites requêtes, l'analyse fait des calculs ponctuels et lourds.

Conséquences

- **Positives**
  - Chaque équipe possède un service, son contrat et sa base.
  - Une panne de la messagerie n'empêche pas de postuler à une offre.
  - Le service d'analyse peut être dimensionné séparément.
- **Négatives / risques**, et comment on les limite
  - Coût d'outillage → monorepo (ADR-0003), Dockerfile et Makefile communs.
  - Dérive des interfaces → Contract First (ADR-0002).
  - Données réparties → références par UUID et dénormalisation des champs d'affichage (ADR-0004).
  - Risque de « monolithe distribué » si les services s'appellent en chaîne pour chaque requête → on surveille le nombre d'appels inter-services (ADR-0006).
- **Suivi** : si l'équipe se réduit ou si le coût d'exploitation devient trop élevé, regrouper des services est possible, car le découpage interne en paquets est identique partout. Le chemin inverse (extraire des services d'un monolithe) aurait été plus coûteux.

ADR-0002 : Approche Contract First avec OpenAPI et SPIs générés

- **Statut** : Accepté
- **Date** : 2026-11-01
- **Décideurs** : équipes back, front
- **Ticket** : AL-2

Contexte

Cinq services et un front sont développés en parallèle par des équipes différentes. Sans contrat stable, chaque équipe interprète les API à sa façon. Les écarts apparaissent alors tard, à l'intégration. Le sujet impose une démarche Contract First.

Options étudiées

1. **Code First** : on écrit les contrôleurs, puis on génère la documentation (springdoc).
    - Avantage : rapide pour un service isolé.
    - Inconvénients : le contrat change à chaque modification de code ; le front attend que le back soit écrit ; rien n'empêche une rupture de compatibilité.
2. **Contract First, contrat seulement documentaire** : on écrit l'OpenAPI, puis on code à la main.
    - Inconvénient : le code et le contrat finissent par diverger, car rien ne les relie.
3. **Contract First avec génération** : l'OpenAPI est la source de vérité. Les interfaces Java (`*Api`) et les modèles sont **générés** (openapi-generator) et les contrôleurs les implémentent.

Décision

Nous retenons l'**option 3**.

- Les contrats sont dans `contracts/*.yaml`, avec un `CHANGELOG.md`.
- Ils sont générés en artefacts Maven `*-spi` versionnés (ADR-0007).
- Chaque contrôleur `implements` l'interface générée. Si le code s'écarte du contrat, **la compilation échoue**.
- Toute modification de contrat passe par une Pull Request : on modifie le contrat, on incrémente la version, on régénère et on aligne les `pom.xml`.
- La CI vérifie que les contrats génèrent des SPIs valides (job `spis-check`).

Conséquences

- **Positives**
    - Le front peut développer contre des mocks générés depuis le contrat, avant que le back existe.
    - La documentation d'API est toujours exacte, puisque c'est elle qui produit le code.
    - Les ruptures de compatibilité sont visibles en revue de code, sur le diff du YAML.
- **Négatives / risques**
    - Il faut connaître OpenAPI et les options du générateur.

    - Une modification de contrat demande plus de cérémonie : version, régénération, PR.
    - Le nommage des classes dépend du YAML. Les schémas inline mal nommés (`inline_response_200`) ont dû être renommés (voir `contracts/CHANGELOG.md`).
- **Suivi** : ajouter un contrôle automatique des ruptures de compatibilité en CI (ex. `openapi-diff`) quand les contrats évolueront après la v1.0.0.

ADR-0003 : Monorepo unique pour les services, les contrats et l'infrastructure

- **Statut** : Accepté
- **Date** : 2026-11-01
- **Décideurs** : équipe devops
- **Ticket** : AL-2

Contexte

Le projet comprend 5 services, des contrats partagés, des SPIs générés, un Dockerfile commun, une infrastructure Kubernetes et des workflows CI/CD. Au départ, l'infrastructure était pensée en dépôts séparés (voir `alumni-uasz-infra/README.md`, « dépôts frères »). L'équipe est petite et les services évoluent souvent en même temps, par exemple quand un contrat change.

Options étudiées

1. **Un dépôt par service** (polyrepo)
   - Avantages : droits et historique séparés ; CI simple par dépôt.
   - Inconvénients : une modification transverse (contrat + service + infra) demande plusieurs PR synchronisées ; les SPIs doivent être publiés sur un registre externe ; la configuration est dupliquée.
2. **Monorepo**
   - Avantages : une modification transverse tient dans **une seule PR, atomique** ; outillage partagé (Dockerfile, Sonar, workflows) ; les SPIs sont accessibles par un chemin relatif ; une seule source de vérité.
   - Inconvénients : la CI doit éviter de tout reconstruire à chaque commit ; le dépôt grossit.

Décision

Nous retenons un **monorepo**. L'indépendance des services est préservée par :

- un dossier autonome par service (son `pom.xml`, son `Makefile`, son `docker-compose.yml`, ses manifestes `k8s/`) ;
- une **CI sélective** (`dorny/paths-filter`) : seuls les services modifiés sont construits et déployés.

Conséquences

- **Positives** : un changement de contrat, sa régénération et son adoption par les services passent en une seule revue. L'outillage n'est écrit qu'une fois.
- **Négatives / risques**
  - Tout le monde a accès à tout le code → la discipline de revue par PR remplace la séparation des droits.
  - Le README d'`alumni-uasz-infra` parle encore de dépôts séparés → à mettre à jour.
- **Suivi** : si le nombre de services ou d'équipes augmente fortement, réévaluer (CODEOWNERS par dossier dans un premier temps).

ADR-0004 : Une base PostgreSQL par service, schéma géré par Flyway

- **Statut** : Accepté
- **Date** : 2026-11-01
- **Décideurs** : équipes back, devops
- **Ticket** : AL-2

Contexte

Les microservices (ADR-0001) n'apportent l'autonomie promise que si chacun maîtrise ses données. Une base partagée crée un couplage caché : la modification d'une table par une équipe casse le service d'une autre. Les données sont relationnelles (utilisateurs, offres, candidatures, messages) et demandent de l'intégrité transactionnelle.

Options étudiées

1. **Une base partagée par tous les services** : simple et permet les jointures, mais crée un couplage fort par le schéma et un point de défaillance unique.
2. **Un schéma par service dans une même instance** : isolation logique, moins de ressources, mais l'isolation repose sur la discipline et l'instance reste partagée.
3. **Une base par service** (*database per service*).

Pour le SGBD : PostgreSQL (open source, robuste, types riches comme `uuid`, `jsonb` et les tableaux), MySQL, MongoDB (moins adapté à un modèle relationnel avec contraintes).

Décision

- **Une base PostgreSQL 16 par service**. Seul le service propriétaire y accède.
- Les données d'un autre service sont **référencées par UUID** (`userId`, `companyId`, `offerId` …), sans clé étrangère entre bases.
- Les champs d'affichage nécessaires sont **dénormalisés** (`companyName`, `studentName`) pour éviter des appels réseau à chaque lecture.
- Le schéma est **versionné par Flyway** (`db/migration/V1__…`). On n'utilise jamais `ddl-auto: update` : le mode est `validate`. Une migration déjà appliquée n'est jamais modifiée.

Conséquences

- **Positives** : les services évoluent indépendamment ; le schéma est reproductible dans chaque environnement ; les ressources sont dimensionnées par base.
- **Négatives / risques**
  - Pas de jointure ni de transaction entre services. Les rapports transverses passent par `analytics-matching-service`, qui agrège via les API.
  - Les données dénormalisées peuvent devenir obsolètes (une entreprise renommée, par exemple). Il faut un mécanisme de mise à jour : appel ou événement (ADR-0006).
  - La suppression d'un utilisateur (RGPD) doit être propagée à chaque service.

    - 5 bases à sauvegarder et à superviser.
- **Suivi** : en production, envisager une instance PostgreSQL managée avec une base par service, ce qui réduit le coût d'exploitation.

ADR-0005 : Authentification stateless par JWT signé avec un secret partagé (HS256)

- **Statut** : Accepté, à réévaluer avant la mise en production
- **Date** : 2026-11-01
- **Décideurs** : équipes back, devops
- **Ticket** : AL-2

Contexte

Les 5 services exposent des API protégées et doivent connaître l'identité et le rôle de l'appelant (`ETUDIANT` , `ALUMNI` , `ENSEIGNANT` , `ADMINISTRATEUR`). Ils tournent en plusieurs réplicas sans état (ADR-0008). Le cahier des besoins (AL-62) exige des JWT, des mots de passe hachés et un contrôle d'accès par rôle (RBAC).

Options étudiées

1. **Sessions HTTP côté serveur** : incompatibles avec des services sans état répliqués, sauf à ajouter un stockage de sessions partagé (Redis).
2. **Validation centralisée** : chaque service appelle `user-service` à chaque requête. Ce service devient alors un point de défaillance unique et ajoute de la latence partout.
3. **JWT HS256, secret partagé** : `user-service` signe, et chaque service vérifie localement avec le même secret.
4. **JWT RS256/ES256 avec publication des clés (JWKS)** : `user-service` signe avec sa clé privée et publie la clé publique. Les services vérifient sans pouvoir signer.
5. **Fournisseur d'identité externe** (Keycloak) : très complet, mais c'est un composant lourd de plus à opérer.

Décision

Pour la v1, nous retenons l'**option 3**.

- `user-service` émet un **access token** court et un **refresh token** révocable (stocké haché en base).
- Chaque service vérifie la signature, l'émetteur (`iss = alumni-uasz-user`) et l'expiration, puis applique les règles de rôle.
- Le secret fait au moins 32 octets (Base64). Il est généré par `make setup` et injecté en `Secret` Kubernetes, jamais commité.
- Les mots de passe sont hachés avec **BCrypt**.

Pourquoi pas RS256 tout de suite : HS256 est plus simple à mettre en place pour démarrer (pas de gestion de paires de clés ni d'endpoint JWKS). La différence de sécurité est connue et assumée pour la v1.

Conséquences

- **Positives** : pas d'appel réseau pour authentifier ; montée en charge horizontale sans session ; `user-service` n'est pas un point de défaillance à l'exécution.
- **Négatives / risques**

    - **Le secret est partagé par 5 services.** S'il fuit depuis n'importe lequel d'entre eux, un attaquant peut **forger** des jetons valides, y compris administrateur. N'importe quel service peut aussi signer.
    - Un access token ne peut pas être révoqué avant son expiration → durée de vie courte (≈ 15 min) et refresh token révocable.
    - Changer de secret impose de redéployer tous les services de façon coordonnée.
- **Suivi** : avant la mise en production, remplacer cette décision par un ADR « JWT RS256 + JWKS » (option 4). Seul `user-service` détiendra alors la clé privée, et la rotation des clés pourra se faire sans redéploiement coordonné.

ADR-0006 : Communication entre services en REST synchrone (pas de broker en v1)

- **Statut** : Accepté, à réévaluer
- **Date** : 2026-11-07
- **Décideurs** : équipes back, devops
- **Ticket** : AL-2

Contexte

Certains flux traversent plusieurs services :

- `job-offer-service` a besoin du profil et du CV d'un étudiant, et doit faire envoyer des notifications ;
- `analytics-matching-service` agrège des étudiants et des offres ;
- `user-service` fait envoyer l'email de confirmation.

Les contrats exposent déjà `POST /notifications/send`.

Options étudiées

1. **REST synchrone** : un service appelle l'API d'un autre en propageant le JWT.
    - Avantages : simple ; réutilise les contrats existants ; facile à tester et à tracer.
    - Inconvénients : couplage temporel (si l'appelé est indisponible, l'appel échoue) ; latences qui s'additionnent.
2. **Messagerie asynchrone** (RabbitMQ ou Kafka) : les services publient des événements (`ApplicationSubmitted`, `OfferPublished`, `UserDeleted` …) et les autres s'y abonnent.
    - Avantages : découplage ; les notifications ne bloquent pas le flux métier ; propagation naturelle (RGPD, dénormalisation).
    - Inconvénients : un composant de plus à opérer ; cohérence à terme ; contrats d'événements à définir (AsyncAPI) ; débogage plus difficile.

Décision

Pour la v1, nous retenons le **REST synchrone**, avec des garde-fous :

- les **délais d'attente** (timeouts) sont configurés sur chaque client HTTP ;
- l'envoi de notification est **non bloquant pour le métier** : si `notification-messaging-service` échoue, la candidature est quand même enregistrée et l'erreur est journalisée ;
- pas d'appels en chaîne de plus de 2 niveaux pour une requête utilisateur.

Conséquences

- **Positives** : livraison plus rapide ; un seul style d'interface (OpenAPI) à maîtriser.
- **Négatives / risques**
    - Une notification peut être perdue si le service de notification est indisponible.
    - La disponibilité d'un flux dépend de celle de chaque service appelé.
- **Suivi**, critères qui déclencheront un ADR « messagerie asynchrone » :

- des notifications perdues sont constatées ;
- il faut propager la suppression d'un utilisateur (RGPD) ou synchroniser les données dénormalisées ;
- les latences de `analytics-matching-service` deviennent excessives.

ADR-0007 : Distribution des SPIs par un dépôt Maven versionné dans Git

- **Statut** : Accepté
- **Date** : 2026-11-01
- **Décideurs** : équipe devops
- **Ticket** : AL-2

Contexte

Les SPIs générés depuis les contrats (ADR-0002) doivent être consommables par chaque service, en local comme en CI et lors du build Docker. On veut éviter qu'un développeur ait à régénérer les SPIs lui-même, et garantir que tout le monde compile contre la même version.

Options étudiées

1. **Génération dans chaque service** (plugin openapi-generator dans chaque `pom.xml`) : pas d'artefact à publier, mais la configuration est dupliquée 5 fois, chaque build régénère, et rien ne garantit que deux services utilisent la même version du contrat.
2. **Registre Maven externe** (GitHub Packages, Nexus) : c'est la solution standard en entreprise, mais chaque développeur et chaque build Docker doivent s'authentifier, et il faut un compte ou un serveur à administrer.
3. **Dépôt Maven fichier versionné dans Git** (`spis/repo/`) : les artefacts publiés sont commités, et les services les lisent via `../spis/repo`.

Décision

Nous retenons l'**option 3**.

- `spis/build.sh` génère et publie dans `spis/repo/`. Le résultat est commité avec le contrat.
- Les versions sont immuables : toute modification impose une nouvelle version (`updatePolicy=never` côté consommateurs).
- Pour consommer une nouvelle version : `git pull`, puis aligner `alumni.spi.version`.

Conséquences

- **Positives** : aucune authentification ni serveur ; build Docker hors ligne (le Dockerfile copie `spis/repo/`) ; la version du contrat et le code sont dans le même commit.
- **Négatives / risques**
  - Des binaires (`.jar`) sont versionnés dans Git, ce qui fait grossir le dépôt à chaque version.
  - On peut oublier de régénérer après une modification du contrat → le job CI `spis-check` le détecte.
- **Suivi** : passer sur GitHub Packages si le nombre de versions fait trop grossir le dépôt, ou si des consommateurs extérieurs au monorepo apparaissent.

ADR-0008 : Conteneurisation Docker et déploiement Kubernetes (kustomize, kind en local)

- **Statut** : Accepté
- **Date** : 2026-11-01
- **Décideurs** : équipe devops
- **Ticket** : AL-2

Contexte

Cinq services et leurs bases doivent tourner de façon identique sur les postes des développeurs, en CI et en production. Le cahier des besoins exige de la disponibilité et de l'évolutivité, et le sujet impose Docker et la CI/CD.

Options étudiées

1. **docker-compose seul**, y compris en production : simple, mais pas de rolling update, pas de redémarrage sur sonde ni de répartition sur plusieurs nœuds.
2. **Kubernetes** : réplicas, rolling updates, sondes de santé, gestion des secrets, ingress. Plus complexe, mais c'est le standard du marché.
3. **PaaS** (Heroku, Render…) : très simple, mais dépendance à un fournisseur, coût, et peu de valeur pédagogique.

Pour gérer les manifestes : Helm (moteur de templates puissant mais verbeux) ou kustomize (intégré à `kubectl`, overlays simples).

Décision

- **Docker** : un `docker/service.Dockerfile` unique, multi-étapes, paramétré par `SERVICE`, avec un JRE 21 et un utilisateur non root (UID 10001).
- **Développement local** : `docker-compose` par service (le service et son PostgreSQL), piloté par `make`.
- **Orchestration** : Kubernetes avec **kustomize**. Chaque service a une `base` et des overlays `dev` et `prod`. En local, on utilise **kind**, avec un ingress NGINX qui route par préfixe.
- **Par défaut** : 2 réplicas, `RollingUpdate` avec `maxUnavailable: 0`, sondes liveness/readiness Actuator, requêtes et limites de ressources.
- **CD** : les images sont publiées sur GHCR, puis on fait `kubectl apply -k`.

Conséquences

- **Positives** : environnements identiques du poste de développement à la production ; déploiement sans interruption ; une seule image à maintenir.
- **Négatives / risques**
  - Il faut apprendre Kubernetes, et kind consomme des ressources sur le poste.
  - En dev, PostgreSQL tourne en `StatefulSet` dans le cluster, ce qui ne convient pas tel quel à la production (sauvegardes, haute disponibilité).
- **Suivi** : choisir l'hébergement de production (cluster managé et base managée) dans un ADR dédié.

ADR-0009 : Java 21 et Spring Boot comme stack backend commune

- **Statut** : Accepté
- **Date** : 2026-11-01
- **Décideurs** : équipe back
- **Ticket** : AL-2

Contexte

Les microservices permettent techniquement d'utiliser un langage différent par service. Mais l'équipe est petite, les membres passent d'un service à l'autre, et l'outillage (Dockerfile, Sonar, CI, SPIs) doit rester commun.

Options étudiées

1. **Java 21 + Spring Boot** : écosystème complet (Web, Security, Data JPA, Validation, Actuator) ; enseigné dans le cursus ; très demandé en entreprise ; openapi-generator le supporte bien.
2. **Node.js (NestJS)** : rapide à démarrer, même langage que le front, mais typage et ORM moins mûrs pour un domaine transactionnel, et moins maîtrisé par l'équipe back.
3. **Plusieurs langages selon le service** (ex. Python pour le matching) : chacun utilise l'outil le plus adapté, mais l'outillage est multiplié et les membres deviennent moins interchangeables.

Décision

Java 21 (LTS) et Spring Boot 4.1 pour les 5 services, avec Maven (wrapper `mvnw`), Spring Security, Data JPA, Validation, Actuator, Flyway et PostgreSQL. La structure de paquets est la même partout (`controller`, `service`, `repository`, `model`, `dto`, `security`, `config`, `exceptions`).

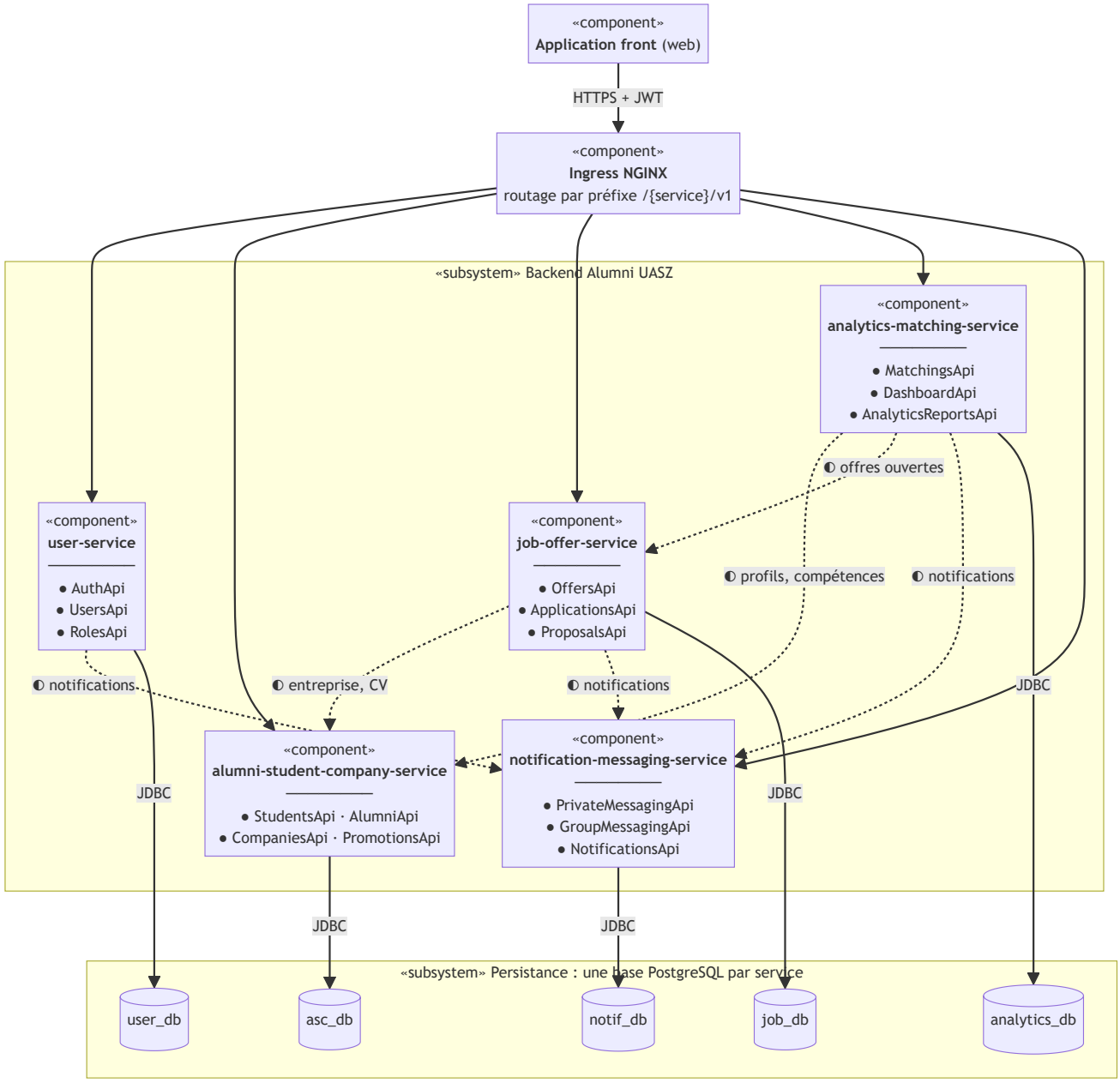
Conséquences

- **Positives** : un développeur passe d'un service à l'autre sans changer de repères ; outillage unique (un Dockerfile, une config Sonar) ; Java 21 est supporté longtemps.
- **Négatives / risques**
  - Une JVM consomme plus de mémoire par service (≈ 384–512 Mi par pod).
  - Spring Boot 4 est récent : certaines bibliothèques tierces peuvent ne pas encore être compatibles.
- **Suivi** : si le matching demande des traitements de données avancés (ML), un service Python dédié pourra être ajouté derrière le même mécanisme de contrat.

Partie III — Diagrammes UML exigés

Diagramme de composants

Composants logiciels de la plateforme et interfaces (fournies ●, requises ◐) entre eux. Chaque interface fournie correspond à un contrat OpenAPI de `contracts/`.



Lecture

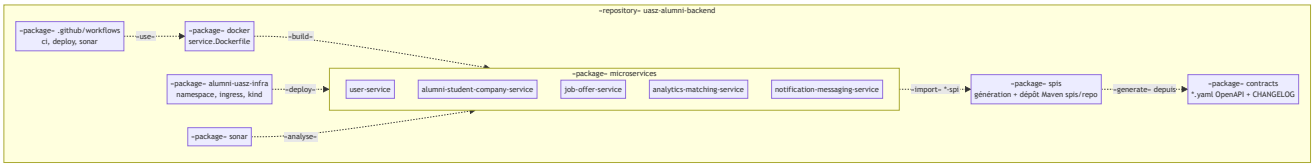
Élément	Signification
● dans un composant	Interface **fournie** par le composant : interfaces Java `*Api` générées depuis le contrat
◐ « notifications »	Appel de `POST /notifications/send`
Flèche pleine	Appel REST entrant via l'ingress, ou accès JDBC à la base du service

Élément	Signification
Flèche pointillée ◐	Interface **requise** : appel REST synchrone de service à service, avec propagation du JWT
Interfaces `*Api`	Générées depuis `contracts/*.yaml` dans les artefacts Maven `*-spi:1.0.0` et implémentées par les contrôleurs de chaque composant (voir le diagramme de packages)

Le JWT émis par `user-service` est vérifié **localement** par chaque service avec un secret partagé. Il n'y a donc pas de dépendance à `user-service` à l'exécution pour l'authentification.

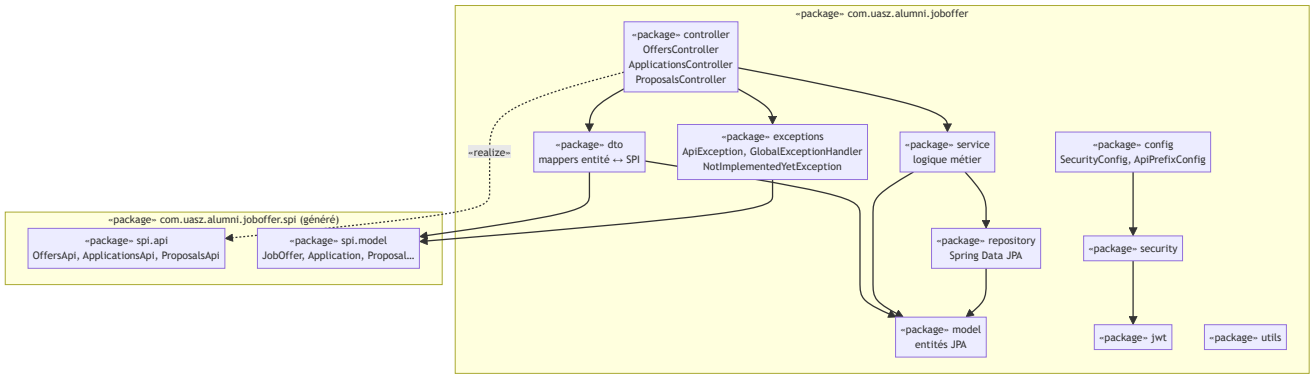
Diagramme de packages

Vue d'ensemble du monorepo



Structure interne d'un microservice

Tous les services ont la même structure. Exemple : `job-offer-service` (`com.uasz.alumni.joboffer`). Les autres racines sont `...user`, `...alumnistudentcompany`, `...analyticsmatching` et `...notificationmessaging`.

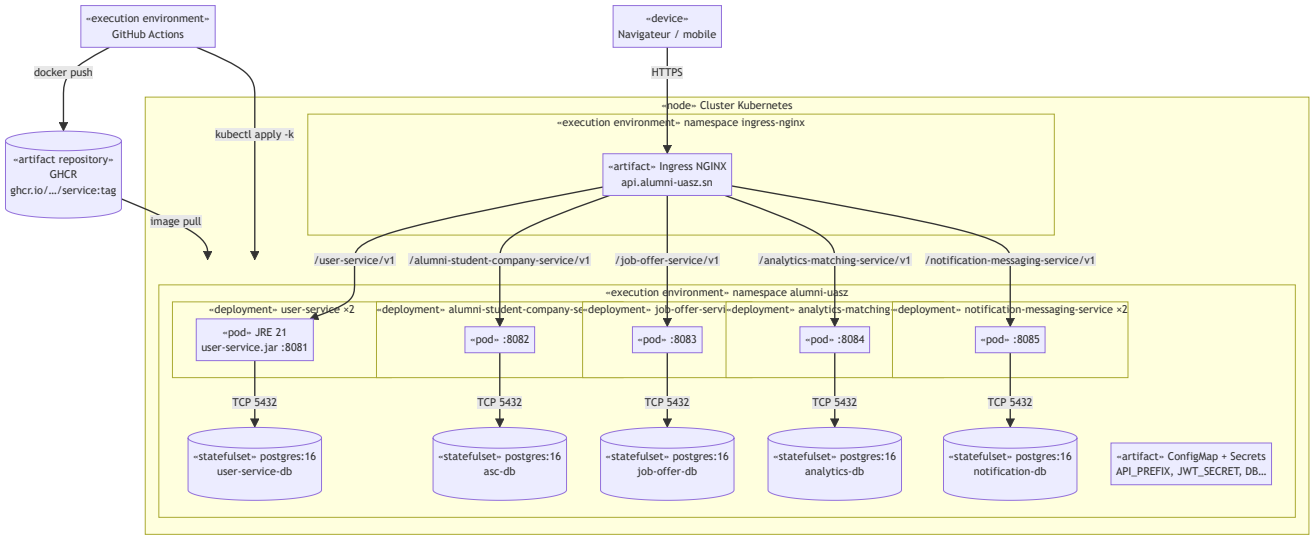


Règles de dépendance

- `controller` ne dépend que de `service`, de `dto` et des interfaces SPI. Il n'accède jamais au `repository`.
- Les entités (`model`) ne sortent jamais du service. On expose uniquement les modèles `spi.model` via `dto`.
- Le package `spi` est **généré**. On ne le modifie pas à la main : on modifie le contrat.

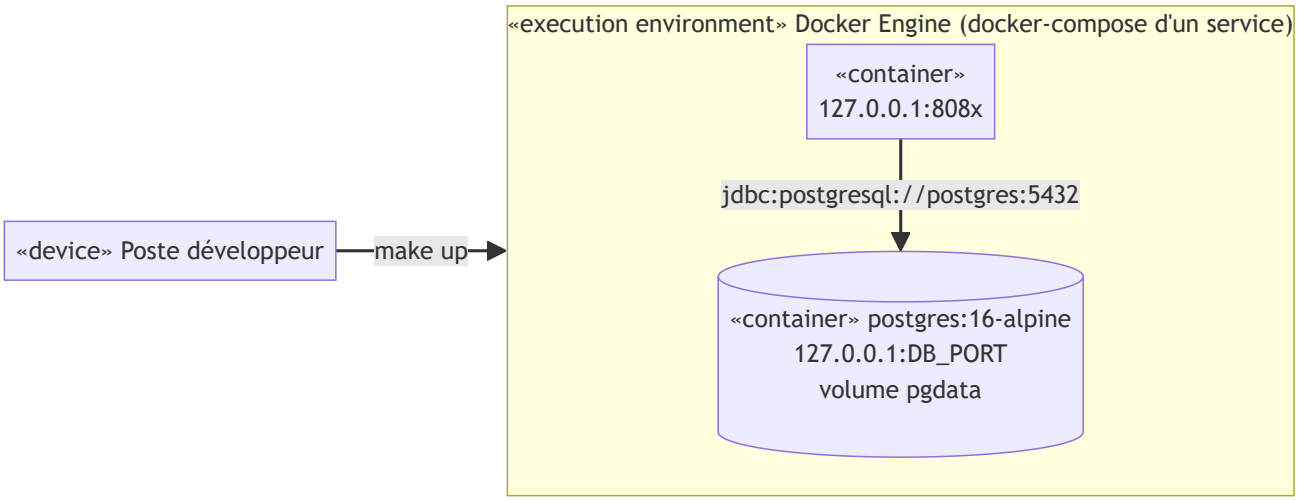
Diagramme de déploiement

Cible Kubernetes (dev avec kind, prod avec le même manifeste)



Élément	Détail (manifestes `*/k8s/`)
Deployment	2 réplicas, `RollingUpdate` (`maxUnavailable: 0`, `maxSurge: 1`), arrêt gracieux de 30 s
Pod	Image `eclipse-temurin` JRE 21, non root (UID 10001), seccomp `RuntimeDefault`, 200m–1 CPU, 384–512 Mi
Sondes	`/actuator/health/liveness` et `/actuator/health/readiness`
Configuration	`ConfigMap` (port, préfixe, CORS, émetteur JWT) ; `Secrets` (`JWT_SECRET`, identifiants de la base)
Base de données	Overlay `dev` : `StatefulSet` `postgres:16-alpine` avec un PVC de 1 Gi par service. En prod : base gérée ou StatefulSet dédié
Ingress	Classe `nginx`, un chemin `Prefix` par service

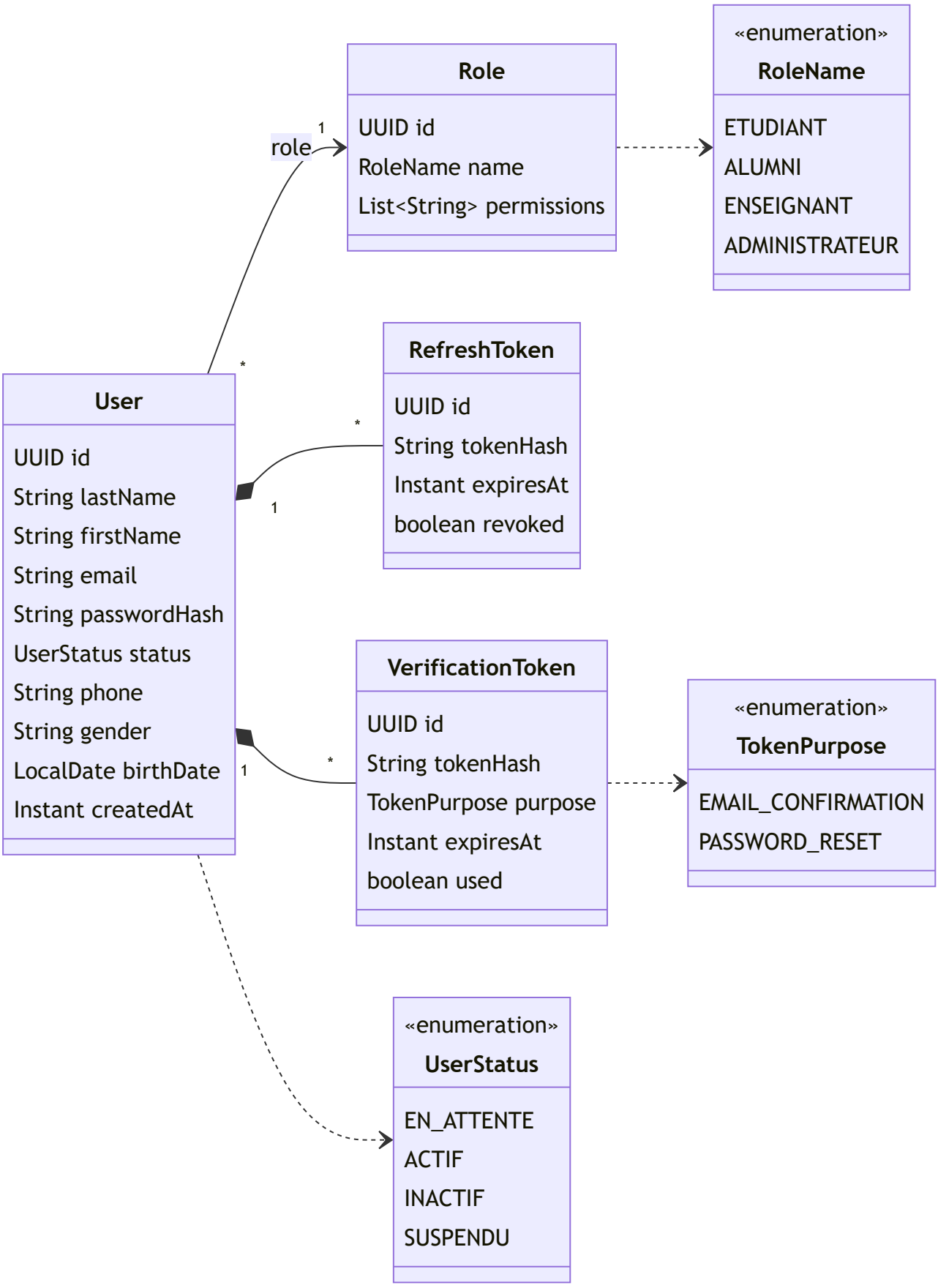
Poste de développement (docker-compose)



Diagrammes de classes (par service)

Modèle de domaine cible de chaque service. Il est dérivé des contrats OpenAPI v1.0.0 et servira de base aux entités JPA (`model/`) et aux migrations Flyway. Les références vers les données d'un **autre service** sont de simples UUID (pas de clé étrangère entre bases). Elles sont marquées `«ref» Service`.

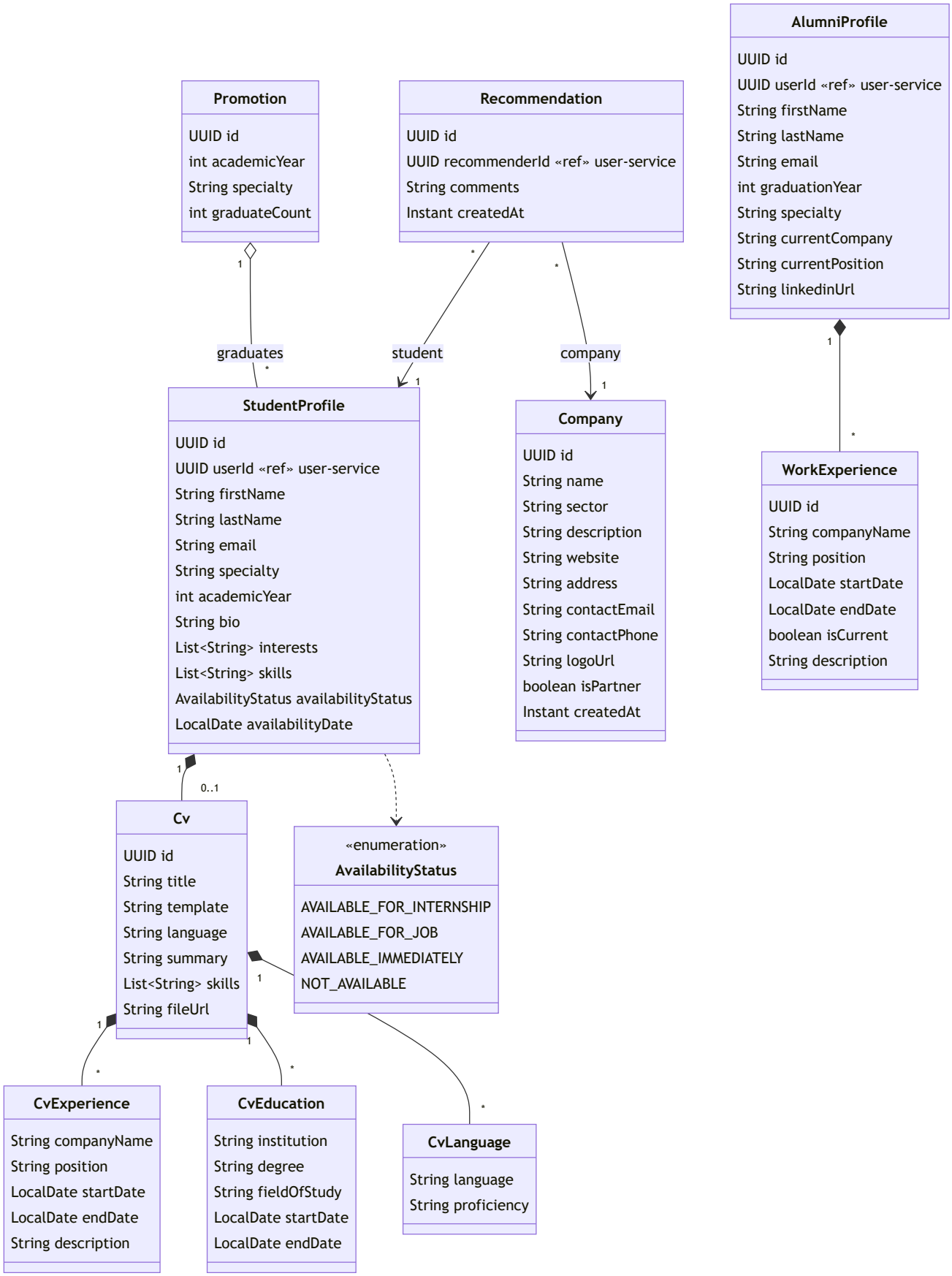
1. user-service



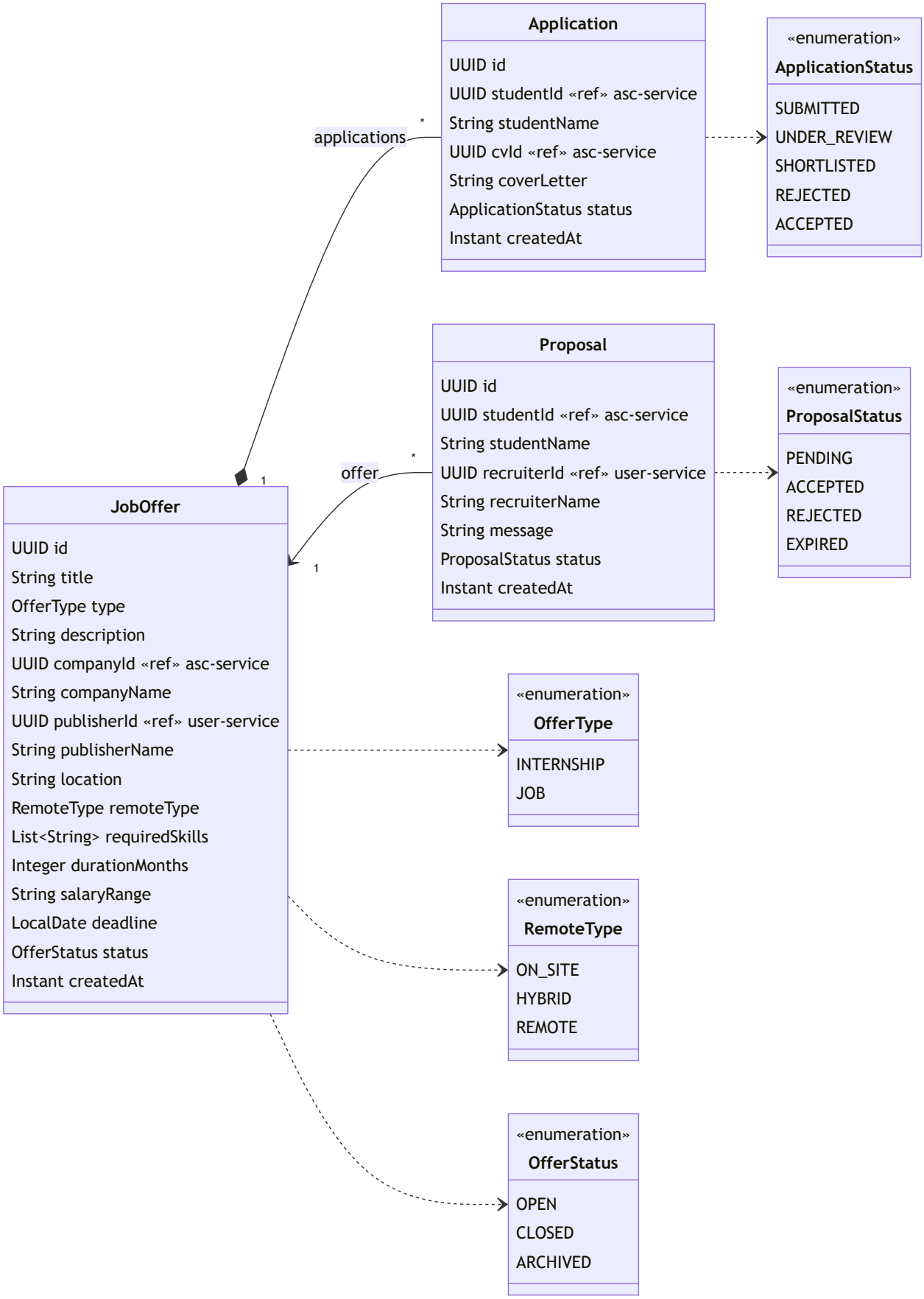
> `RefreshToken`, `VerificationToken` et `passwordHash` ne figurent pas dans le contrat. Ce sont des classes internes, nécessaires aux opérations `/auth/refresh-token`, `/auth/logout`, `/auth/confirm-email` et

> `/auth/reset-password`.

2. alumni-student-company-service

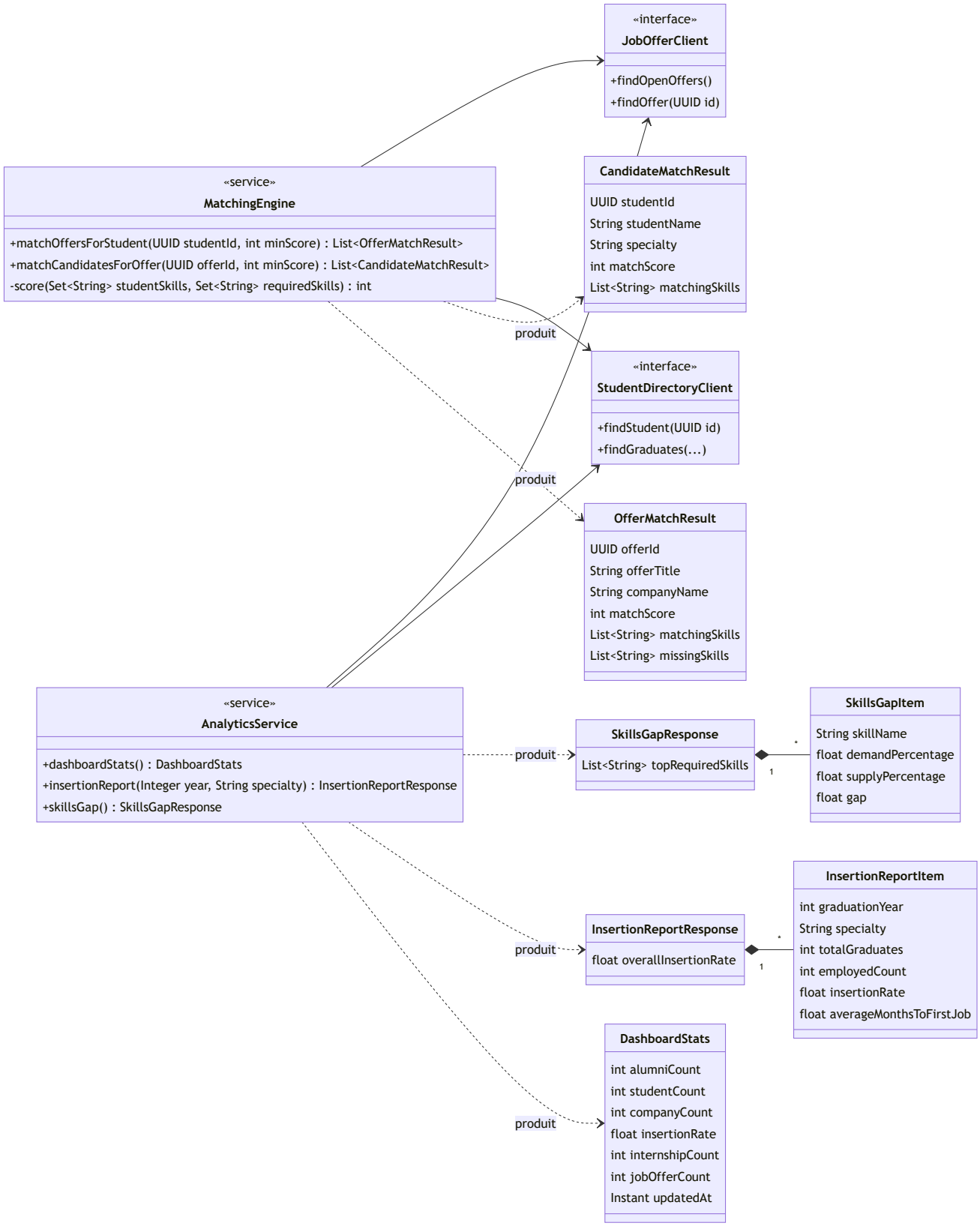


3. job-offer-service

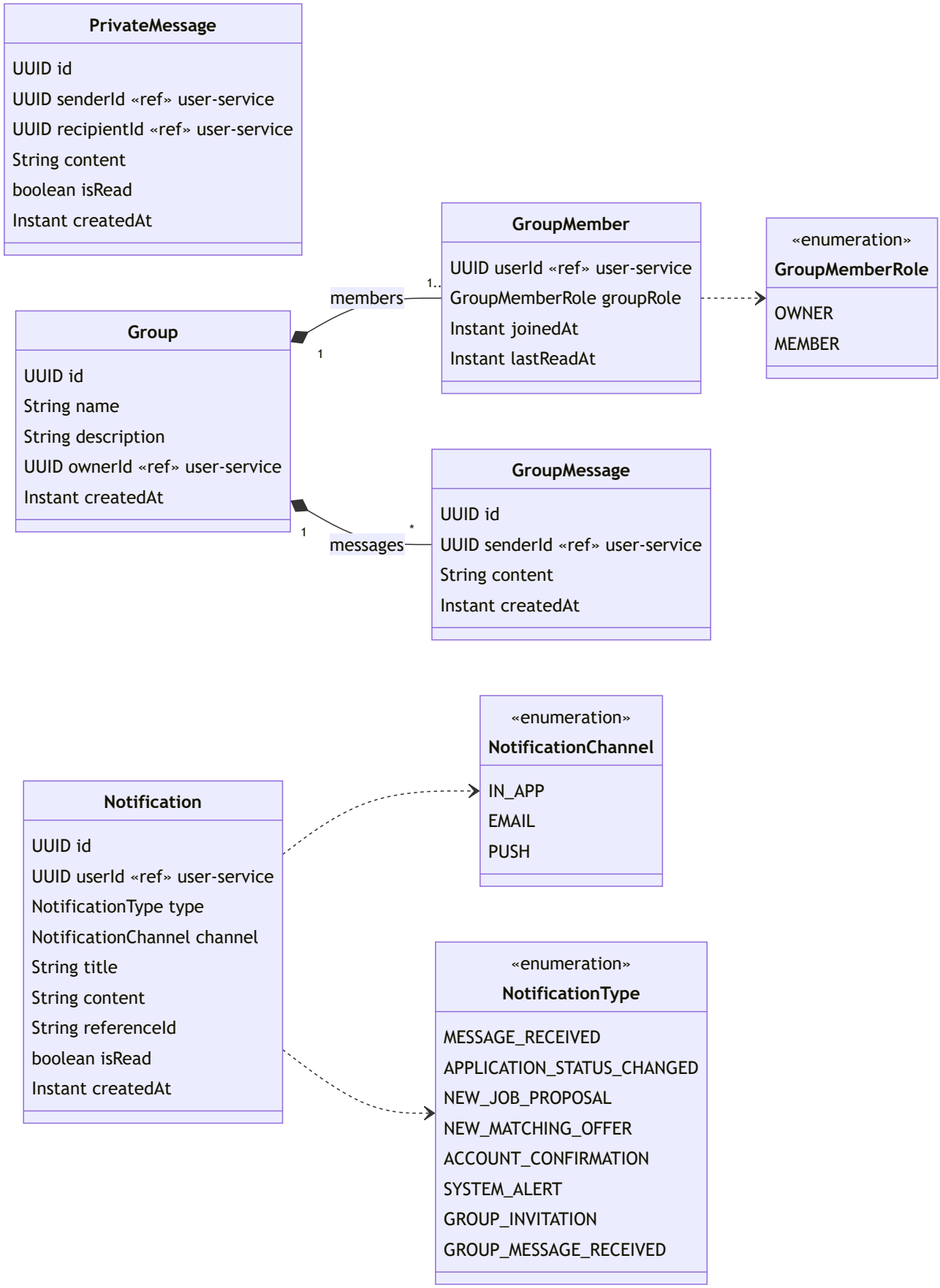


4. analytics-matching-service

Ce service calcule ses résultats à partir des données des autres services. Ses classes sont surtout des **objets de calcul** (non persistés) et un éventuel cache de statistiques.



5. notification-messaging-service



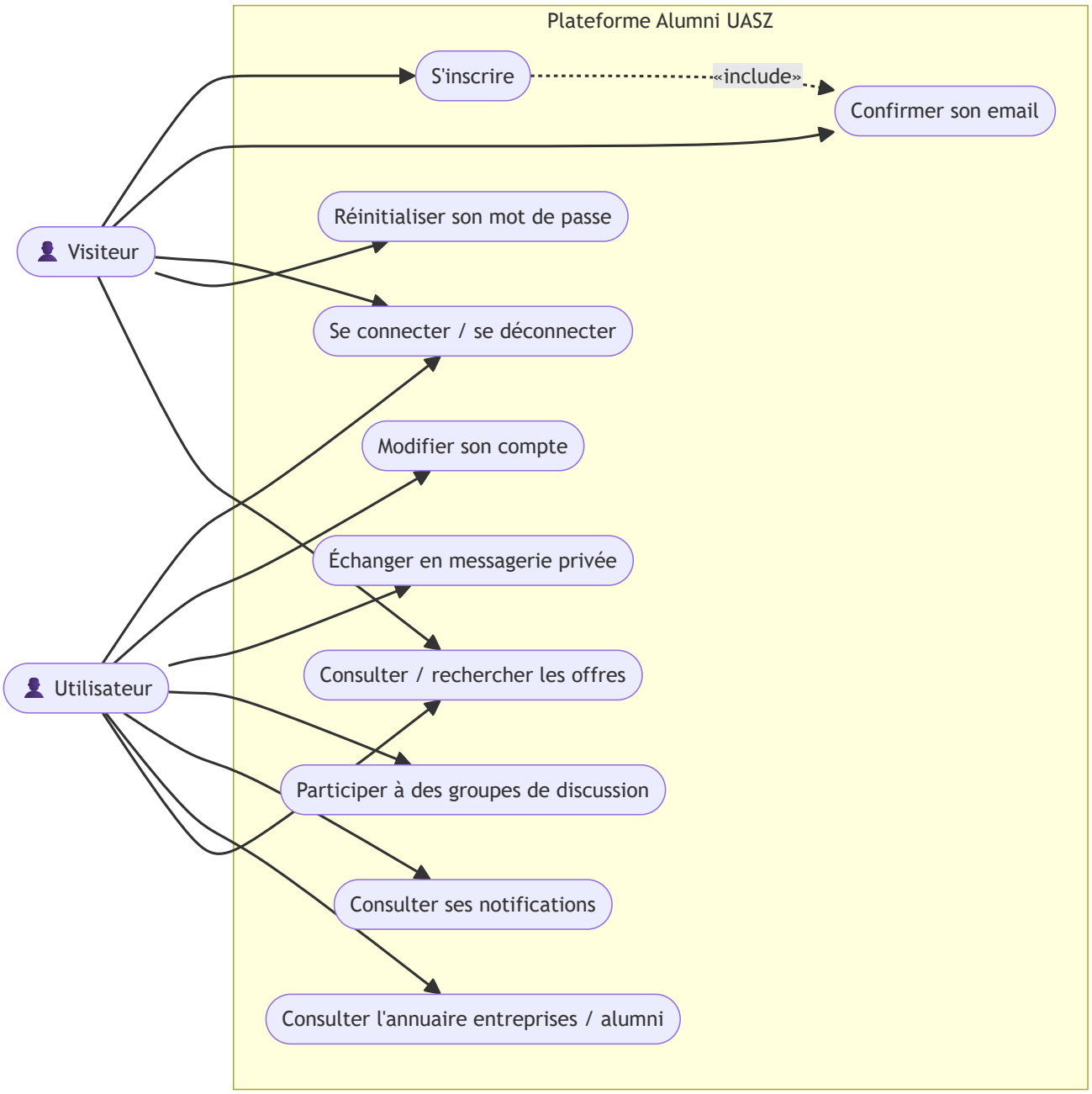
> `GroupMember.lastReadAt` est interne. Il sert à calculer `GroupSummary.unreadCount` et `PATCH /messages/groups/{groupId}/read`. `ConversationSummary` est une vue calculée sur `PrivateMessage`.

Partie IV — Diagrammes complémentaires

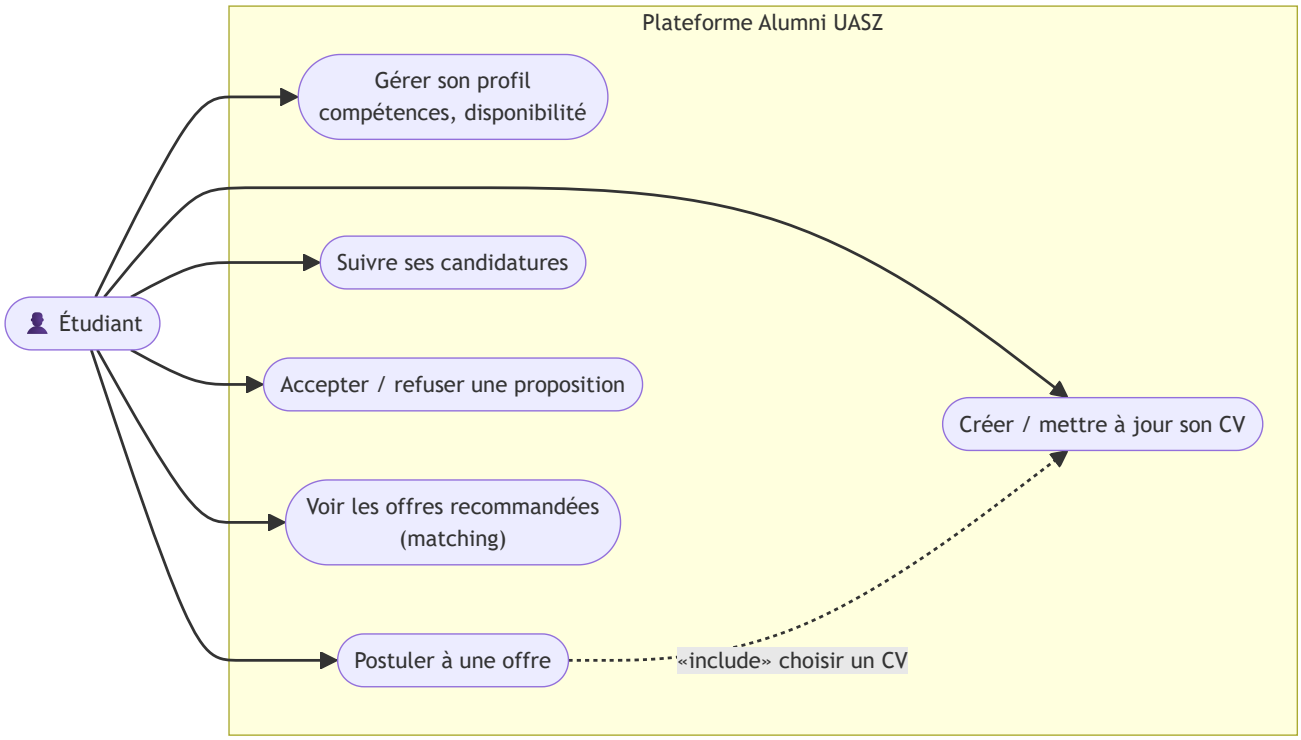
Diagrammes de cas d'utilisation (par rôle)

Rôles définis dans `RoleName` (`user-service`) : **ETUDIANT**, **ALUMNI**, **ENSEIGNANT**, **ADMINISTRATEUR**. L'acteur **Entreprise partenaire** (cahier des besoins, AL-62) n'a pas encore de rôle propre dans le contrat (voir architecture.md, §8).

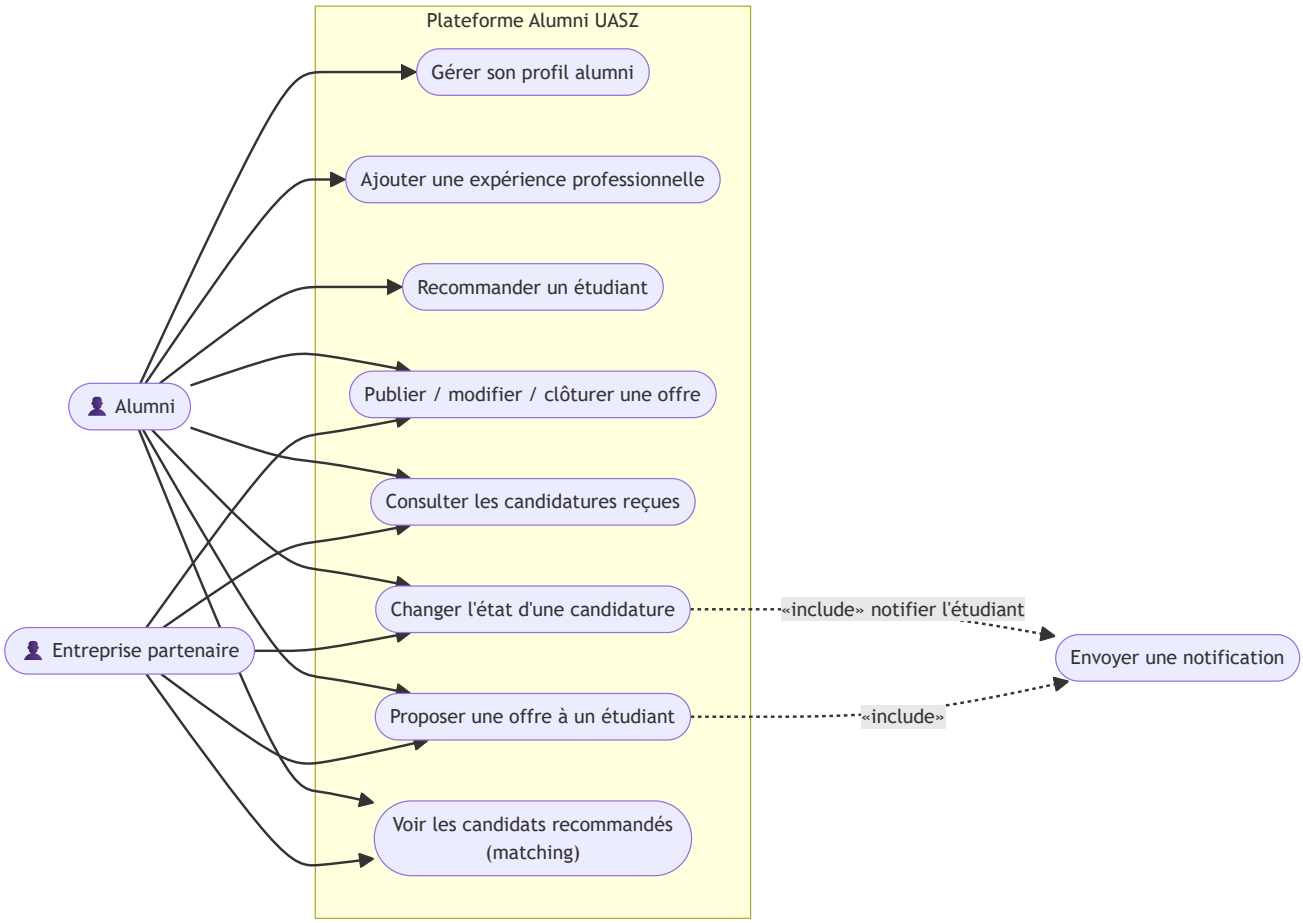
Cas communs à tous les utilisateurs authentifiés



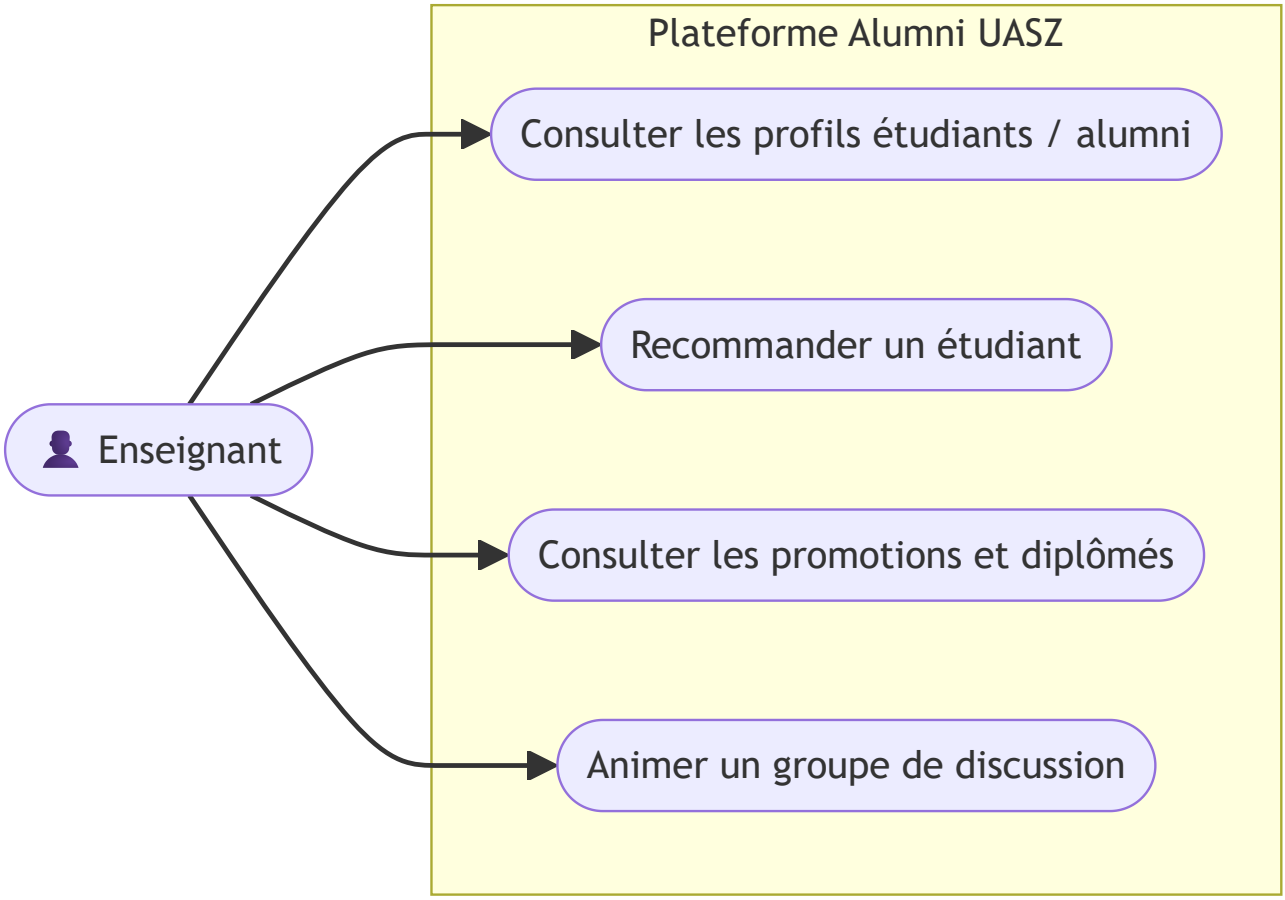
Étudiant



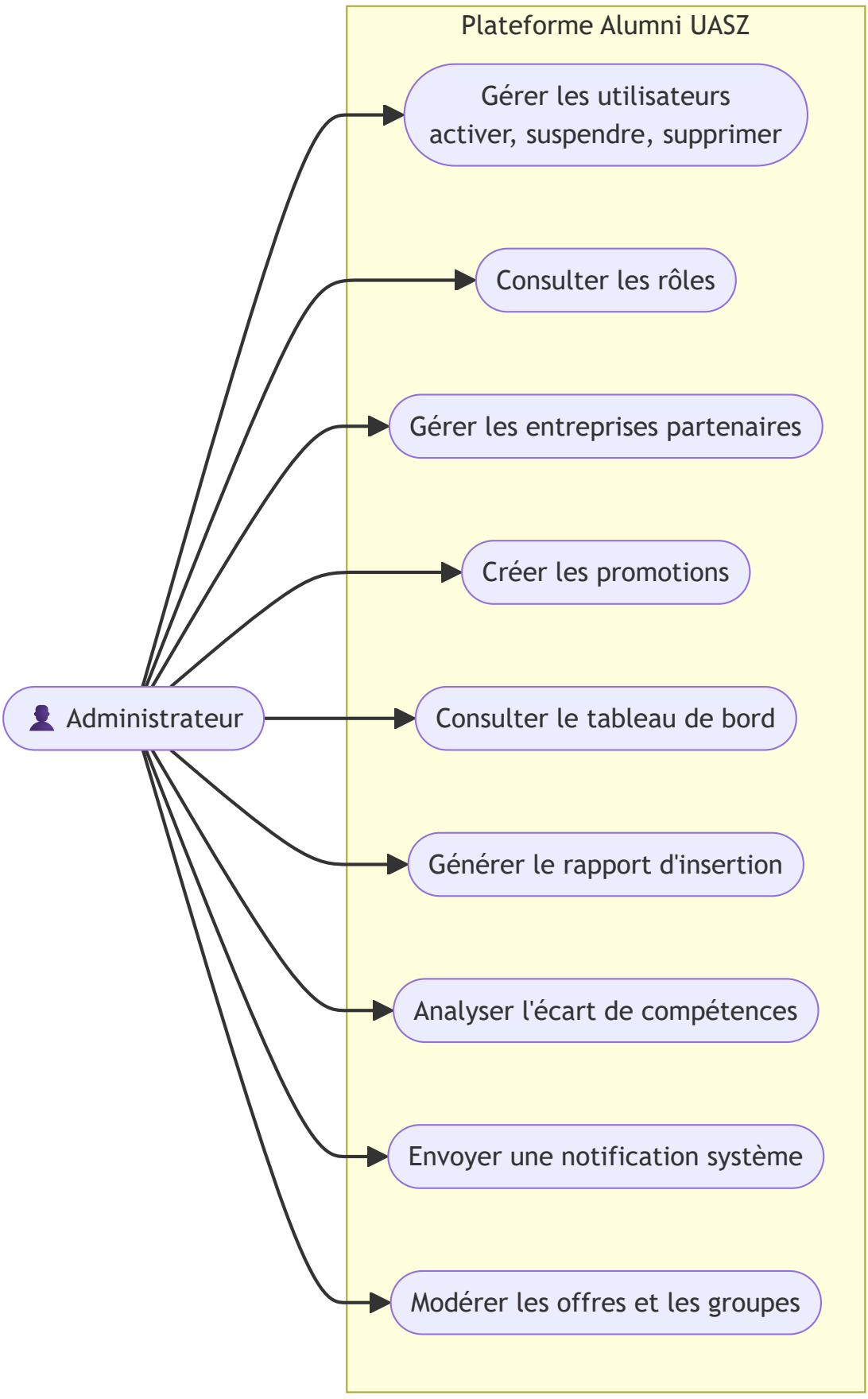
Alumni et Entreprise partenaire



Enseignant



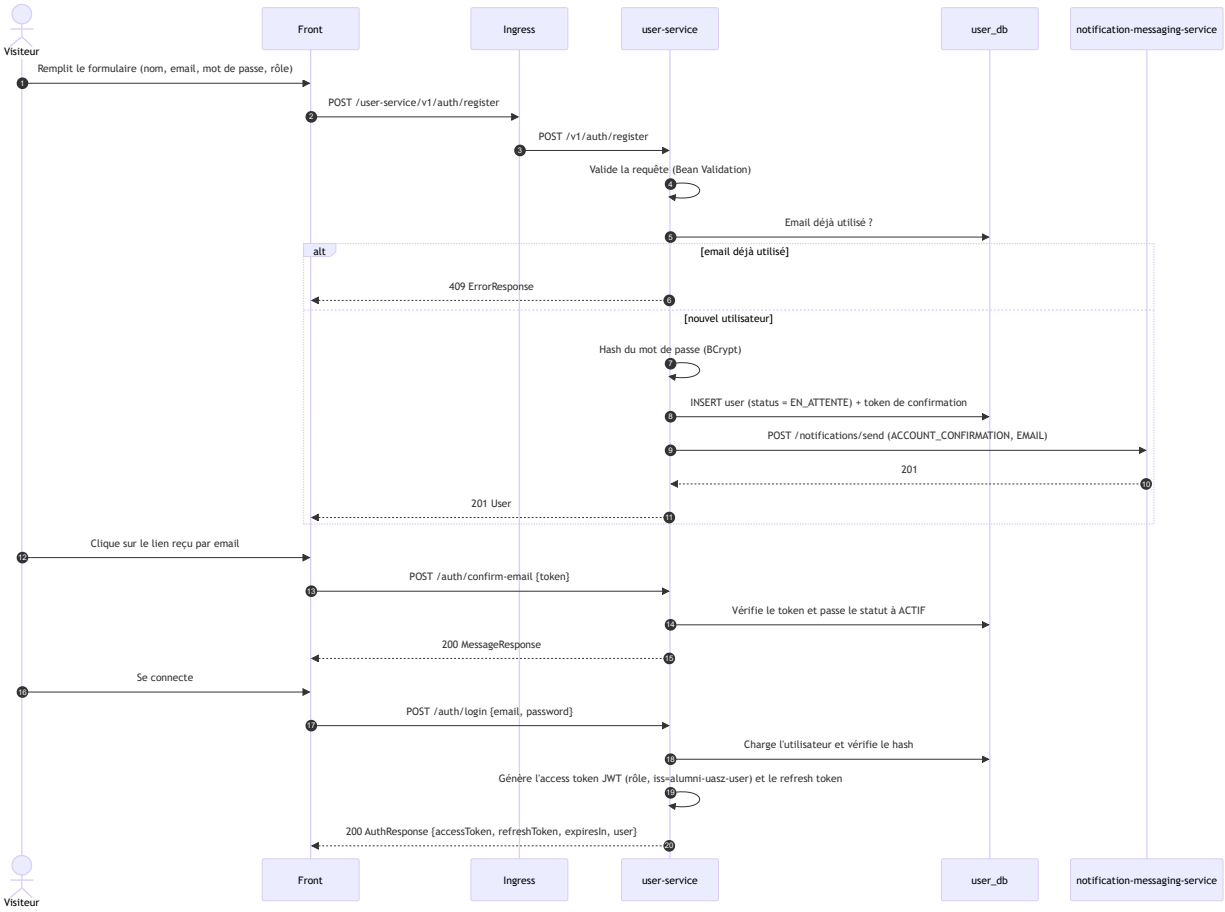
Administrateur


Plateforme Alumni UASZ
Gérer les utilisateurs
activer, suspendre, supprimer
Consulter les rôles
Gérer les entreprises partenaires
Créer les promotions
Administrateur
Consulter le tableau de bord
Générer le rapport d'insertion
Analyser l'écart de compétences
Envoyer une notification système
Modérer les offres et les groupes

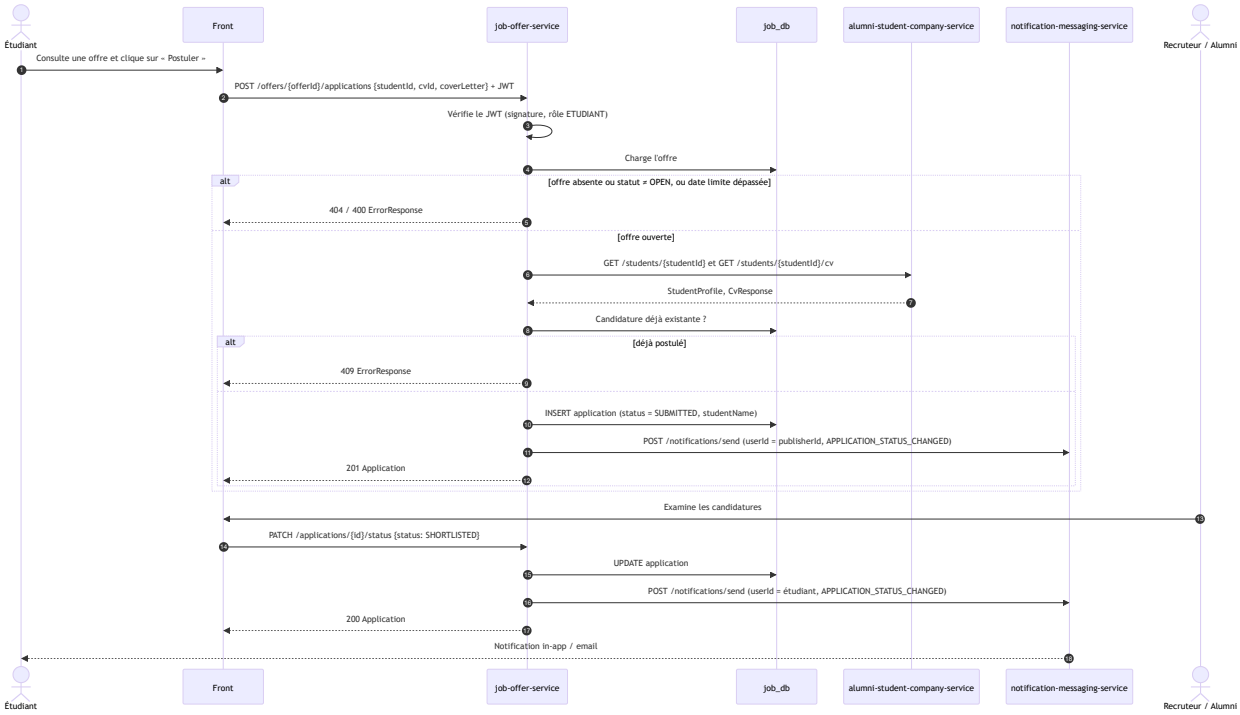
Diagrammes de séquence des flux clés

Flux cibles décrits à partir des contrats OpenAPI. Les appels de service à service sont en REST synchrone, avec propagation du JWT de l'appelant.

1. Inscription et activation du compte



2. Candidature à une offre



3. Matching étudiant → offres

